

Exercício – Padrão Decorator (Segurança)

Objetivo

Você está desenvolvendo um sistema responsável por executar comandos SQL. Inicialmente, o sistema possui apenas uma classe simples que executa queries. Porém, novas necessidades surgiram:

- registrar tentativas de execução
- auditar o usuário responsável
- medir o tempo da operação
- validar possíveis comandos perigosos

Para adicionar esses comportamentos sem modificar a classe principal, utilize o padrão **Decorator**. Implemente um executor SQL utilizando o padrão **Decorator**, permitindo adicionar comportamentos extras de forma flexível e combinável.

SQLInjection

Uma das principais vulnerabilidades em sistemas de software. Simples e perigosa, faz com que pessoas mal-intencionadas consigam invadir sistemas por meio de “injeção” de comandos SQL em consultas parametrizáveis. Assim, o invasor consegue acesso ao sistema através da manipulação de entradas de usuário, executando consultas não autorizadas, acessando dados sensíveis, alterando informações ou até comprometendo completamente o banco de dados da aplicação.

Solução Proposta

Interface principal

```
public interface QueryExecutor {  
    void execute(String sql);  
}
```

Implementação base

```
public class BasicQueryExecutor implements QueryExecutor {  
  
    @Override  
    public void execute(String sql) {  
        System.out.println("Query executada no banco: " + sql);  
    }  
}
```

Base Decorator

Crie o Decorator Base implementando a classe abstrata chamada **QueryExecutorDecorator**. Essa classe deve:

- implementar a interface QueryExecutor
- possuir uma referência para outro QueryExecutor
- delegar a execução para o objeto encapsulado

Decorators

1. Crie um decorator chamado **LoggingDecorator** que deve registrar toda tentativa de execução antes de delegar a query.

Exemplo:

```
executor.execute("DROP TABLE users");
```

Saída esperada:

[LOG] Tentando executar query...

2. Crie um decorator chamado **AuditDecorator** que deve registrar qual usuário tentou executar a query. A classe deve receber o nome do usuário no construtor.

Saída esperada:

[AUDIT] Usuário responsável: luciana

3. Crie um decorator chamado **MetricsDecorator**. Ela deve medir o tempo total da operação. A medição deve envolver a chamada ao próximo executor.

Saída esperada:

[METRICS] Tempo de execução: 2ms

4. Crie um decorator chamado **SecurityValidationDecorator** que deve validar a query antes de permitir sua execução.

A query deve ser bloqueada se contiver:

- DROP
- DELETE
- TRUNCATE
- ALTER
- ' OR '1'='1

- --
- ;

Saída esperada em caso de bloqueio:

[SECURITY] Query bloqueada por validação de segurança.

Quando a query for bloqueada, o executor básico não deve ser chamado.

Classe principal (Main)

Crie uma classe cliente e monte a cadeia de execução:

```
QueryExecutor executor =
    new LoggingDecorator(
        new AuditDecorator(
            new MetricsDecorator(
                new SecurityValidationDecorator(
                    new BasicQueryExecutor()
                )
            ),
            "luciana"
        )
    );
```

Execute os testes:

```
executor.execute("SELECT * FROM users");
executor.execute("DROP TABLE users; --");
executor.execute("SELECT * FROM users WHERE name = 'admin' OR '1'='1'");
```

Resultados esperados:

```
[LOG] Tentando executar: SELECT * FROM users
Executando SQL: SELECT * FROM users
```

```
[LOG] Tentando executar: DROP TABLE users
Query bloqueada: palavra perigosa detectada.
```

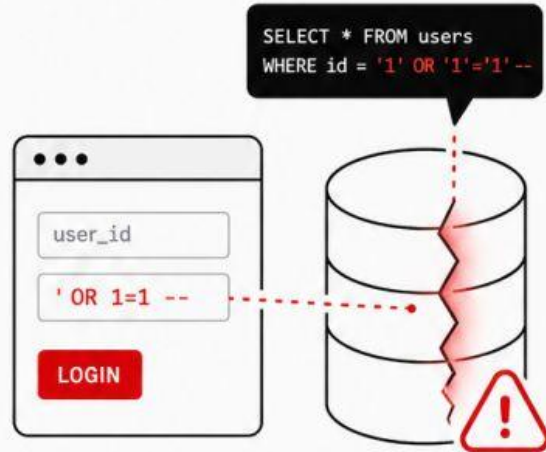
```
[LOG] Tentando executar: SELECT * FROM users WHERE name = '' OR '1'='1'
Query bloqueada: possível SQL Injection detectado.
```

Mais sobre SQLInjection: <https://www.devmedia.com.br/sql-injection/6102>



WHY SQL INJECTION STILL HAPPENS IN 2026

The vulnerability isn't old — our mistakes are.



THE REAL REASONS



1. LEGACY CODE NEVER DIES

- 10–20 year-old systems still running
- Refactoring is expensive; risks stay hidden



2. SPEED > SECURITY CULTURE

- Deadlines drive shortcuts
- Security treated as “later problem”



3. ORM ≠ IMMUNITY

- Raw queries and .execute() bypass protections
- One unsafe endpoint = full compromise



4. INPUT VALIDATION STILL FAILS

- Apps still trust user input incorrectly
- Core issue: data treated as code



5. HIDDEN ATTACK SURFACES

- APIs, microservices, and old endpoints
- Secure frontend, insecure backend



6. AI IS NOT A SILVER BULLET

- AI-generated code may include unsafe patterns
- Developers copy without security review

WHY IT'S STILL DANGEROUS



Full database access



Authentication bypass



Financial & reputational damage



Compliance violations & legal consequences



Remote Code Execution in some cases

REALITY CHECK



Still one of the most common high-severity vulnerabilities



Exploited in real-world breaches in 2025–2026



Exists even in modern frameworks and AI-assisted code

THE FIX IS KNOWN. THE EXECUTION IS MISSING.

THE FIX



PARAMETERIZED QUERIES
Always.



INPUT VALIDATION
Client + Server.



LEAST PRIVILEGE DB ACCESS
Don't over-permit.



SECURE CODE REVIEWS
Humans catch what tools miss.



AUTOMATED TESTING
SAST, DAST, and more.



DON'T TRUST AI BLINDLY
Review, verify, and secure.



**SQL INJECTION ISN'T A TECHNICAL PROBLEM ANYMORE.
IT'S A DISCIPLINE PROBLEM.**

